Namespace Blackjack_Class_Library

Classes

Blackjack

Represents a game of Blackjack, managing the state, player and dealer hands, betting, and game logic.

Card

Represents a playing card with a suit, rank, value, and identifier, and provides methods for creating and shuffling a standard deck.

CardCounter

Provides card counting functionality for a Blackjack game, maintaining running and true counts based on cards dealt to the player and dealer.

FrontEnd

Enums

BlackjackAction

BlackjackResult

BlackjackState

Rank

Suit

Class Blackjack

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

Represents a game of Blackjack, managing the state, player and dealer hands, betting, and game logic.

```
public class Blackjack
```

Inheritance

object ← Blackjack

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Blackjack()

Initializes a new instance of the Blackjack class, setting up the card shoe, player and dealer hands, player credit, bet, and game state.

```
public Blackjack()
```

Properties

currentState

```
public BlackjackState currentState { get; }
```

Property Value

BlackjackState

playerBet

```
public double? playerBet { get; }
```

Property Value

double?

playerCredit

```
public double playerCredit { get; }
```

Property Value

double

playerHand

```
public List<Card> playerHand { get; }
```

Property Value

List<Card>

playerValue

```
public int playerValue { get; }
```

Property Value

int

Methods

CanDoubleDown()

Determines whether the player is eligible to double down based on the current game state, hand size, and bet status.

```
public bool CanDoubleDown()
```

Returns

bool

true if the player can double down; otherwise, false.

DealCards(List<Card>, int)

Deals n cards to either the player or the dealer, specified with toPlayer

```
public List<Card> DealCards(List<Card> toHand, int n = 1)
```

Parameters

`toHand` List<Card>

The hand the cards are dealt to

`n` int

The number of cards to deal

Returns

List<Card>

The hand with cards dealt to it

DealerHandSize()

Gets the number of cards in the dealer's hand.

```
public int DealerHandSize()
```

Returns

int

The count of cards currently held by the dealer.

DealerHandValue()

Calculates and returns the total value of the dealer's hand.

```
public int DealerHandValue()
```

Returns

int

The total integer value of the dealer's hand.

DealerTurn()

Executes the dealer's turn in the Blackjack game, updating the dealer's hand and game state according to standard rules.

```
public void DealerTurn()
```

EndGame()

Ends the current blackjack game, determines the outcome based on player and dealer hands, and outputs the result.

```
public BlackjackResult EndGame()
```

Returns

BlackjackResult

A BlackjackResult value indicating the outcome of the game.

HandToString(List<Card>)

Returns a comma-separated string of card IDs from the given hand.

```
public static string HandToString(List<Card> hand)
```

Parameters

`hand` List<Card>

The list of Card objects representing the hand.

Returns

string

A string containing the IDs of the cards in the hand, separated by commas.

HandValue(List<Card>)

Calculates the total value of a hand of cards, treating aces as 1 or 11 to maximize the hand's value without exceeding 21.

```
public static int HandValue(List<Card> hand)
```

Parameters

`hand` List<Card>

The list of cards whose value is to be calculated.

Returns

int

The total value of the hand.

HiddenDealerCards()

Calculates the number of hidden cards in the dealer's hand.

```
public int HiddenDealerCards()
```

Returns

int

The number of dealer cards not visible to the player.

LookAtDealerHand()

Returns the dealer's hand, hiding the first card during the player's turn.

```
public List<Card> LookAtDealerHand()
```

Returns

List <Card>

A list of cards representing the dealer's hand, with the first card removed if it is the player's turn.

PlaceBet(double)

Places a bet for the player if sufficient credit is available and no game is in progress.

```
public void PlaceBet(double bet)
```

Parameters

`bet` double

The amount to bet.

Exceptions

Exception

Thrown if the bet exceeds available credit, a game is in progress, or the bet amount is zero or negative.

PlayerTurn(BlackjackAction)

Processes the player's action during their turn in a game of Blackjack.

```
public void PlayerTurn(BlackjackAction playerAction)
```

Parameters

`playerAction` BlackjackAction

The action chosen by the player, such as Hit or Stand.

ShoeCount()

Returns the number of cards currently in the shoe.

```
public int ShoeCount()
```

Returns

int

The number of cards in the card shoe.

ShowPlayerCredit()

```
public void ShowPlayerCredit()
```

StartBlackjack()

Initializes and starts a new game of blackjack, dealing cards to the player and dealer and updating the game state.

```
public void StartBlackjack()
```

Enum BlackjackAction

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

```
public enum BlackjackAction
```

Fields

`DoubleDown = 2`

`Hit = 0`

`Stand = 1`

Enum BlackjackResult

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

```
public enum BlackjackResult
```

Fields

BothNatural = 2

DealerBust = 4

DealerNatural = 1

DealerWins = 5

PlayerBust = 3

PlayerNatural = 0

PlayerWins = 6

StandOff = 7

Enum BlackjackState

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

```
public enum BlackjackState
```

Fields

`DealerTurn = 2`

`GameEnd = 3`

`NoGame = 0`

`PlayerTurn = 1`

Class Card

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

Represents a playing card with a suit, rank, value, and identifier, and provides methods for creating and shuffling a standard deck.

```
public class Card
```

Inheritance

object ← Card

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Card(Suit, Rank)

Initializes a new instance of the Card class with the specified suit and rank, setting its value and identifier.

```
public Card(Suit _suit, Rank _rank)
```

Parameters

`_suit` Suit

The suit of the card.

`_rank` Rank

The rank of the card.

Properties

id

```
public string id { get; }
```

Property Value

string

rank

```
public Rank rank { get; }
```

Property Value

Rank

suit

```
public Suit suit { get; }
```

Property Value

Suit

value

```
public int value { get; }
```

Property Value

int

Methods

GetDeck()

Creates and returns a standard 52-card deck with all suits and ranks.

```
public static Card[] GetDeck()
```

Returns

Card[]

An array containing all 52 unique cards in a standard deck.

ShuffleDeck(Card[])

Randomizes the order of cards in the specified deck.

```
public static Card[] ShuffleDeck(Card[] deck)
```

Parameters

`deck` Card[]

The array of cards to shuffle.

Returns

Card[]

The shuffled array of cards.

Class CardCounter

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

Provides card counting functionality for a Blackjack game, maintaining running and true counts based on cards dealt to the player and dealer.

```
public class CardCounter
```

Inheritance

object ← CardCounter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CardCounter(Blackjack)

Initializes a new instance of the CardCounter class with the specified Blackjack game.

```
public CardCounter(Blackjack _game)
```

Parameters

`_game` Blackjack

The Blackjack game instance to associate with the card counter.

Fields

game

Represents the current Blackjack game instance.

```
public Blackjack game
```

Field Value

Blackjack

Properties

decksLeft

```
public double decksLeft { get; }
```

Property Value

double

Methods

GetRunningCount()

Calculates and returns the current running count based on the cards dealt to the player and dealer, resetting the count if the card shoe has been refilled.

```
public int GetRunningCount()
```

Returns

int

The current running count after processing new cards.

GetTrueCount()

Calculates and returns the true count by dividing the running count by the number of decks remaining in the shoe.

```
public double GetTrueCount()
```

Returns

double

The current true count as an integer.

Class FrontEnd

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

```
public static class FrontEnd
```

Inheritance

object ← FrontEnd

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Fields

writeToConsole

```
public static bool writeToConsole
```

Field Value

bool

Methods

Output(string)

```
public static void Output(string str)
```

Parameters

`str` string

Enum Rank

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

```
public enum Rank
```

Fields

`Ace = 0`

`Eight = 7`

`Five = 4`

`Four = 3`

`Jack = 10`

`King = 11`

`Nine = 8`

`Queen = 12`

`Seven = 6`

`Six = 5`

`Ten = 9`

`Three = 2`

`Two = 1`

Enum Suit

Namespace: Blackjack_Class_Library

Assembly: Blackjack_Class_Library.dll

```
public enum Suit
```

Fields

`Club = 0`

`Diamond = 1`

`Heart = 2`

`Spade = 3`